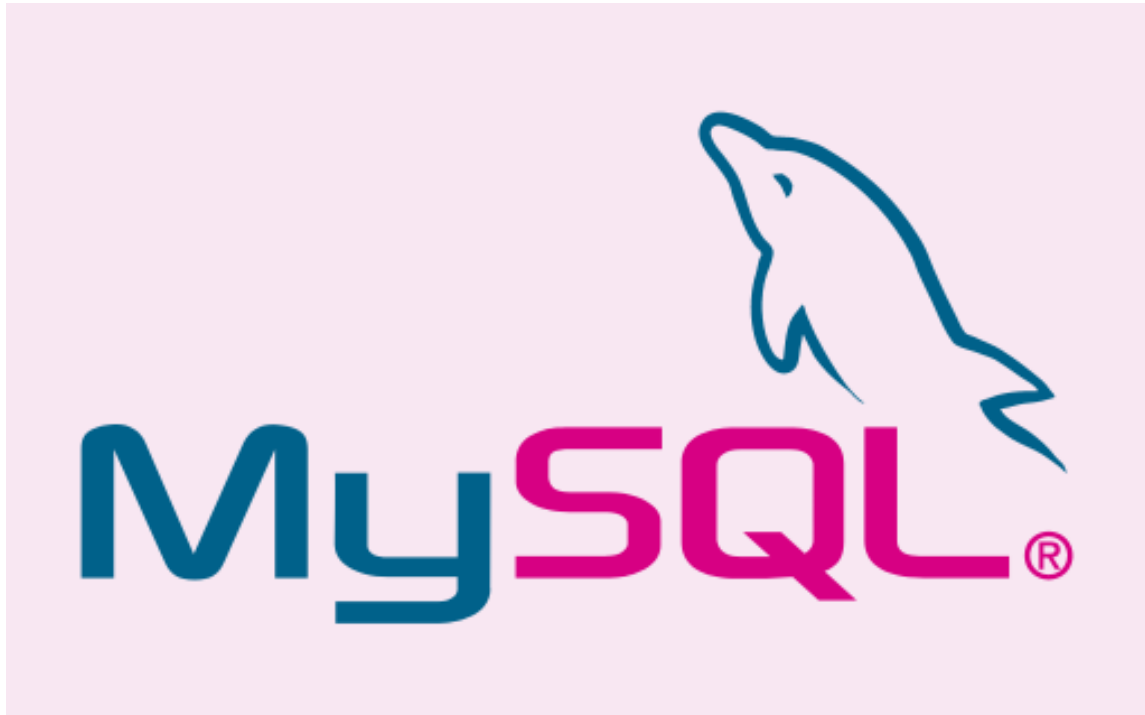


## **Procedimientos en MySQL**



# Índice

---

|                            |          |
|----------------------------|----------|
| <b>Parte práctica.....</b> | <b>3</b> |
| Consulta 1.....            | 3        |
| Consulta 2.....            | 3        |
| Consulta 3.....            | 3        |
| Consulta 4.....            | 4        |
| Consulta 6.....            | 4        |
| Consulta 7.....            | 4        |
| Consulta 8.....            | 5        |
| Consulta 10.....           | 5        |
| Consulta 11.....           | 5        |

## Parte práctica

---

### PROCEDIMIENTOS ALMACENADOS Y FUNCIONES EN MySQL (1 parte)

#### PROCEDIMIENTOS ALMACENADOS:

Antes de hablar de procedimientos en MySQL, hay que indicar como cosa muy importante a tener en cuenta, que a partir del MySQL 5.0, MySQL soporta variables de usuario, las cuales permiten almacenar un valor y hacer referencia a él más tarde; esto posibilita pasar valores de una sentencia a otra.

Una forma de establecer una variable de usuario en MySQL, que es la que nosotros vamos a utilizar de momento, es empleando una sentencia SET, que tiene el siguiente formato:

**SET @nombre\_variable = expresión.**

Con SET, podemos utilizar tanto el signo "=" como el "!=" como operadores de asignación.

La expresión asignada a cada variable puede evaluarse a un valor entero, real, cadena, o NULL.

A la hora de referirnos a una variable tenemos que utilizar:

**@ nombre\_variable.**

2.- Crear una variable denominada @radio que tenga como valor 5. Luego mediante una consulta calcular el área del círculo para ese radio.

```
mysql> SET @radio:= 5;
```

```
mysql> select pi();
```

```
mysql> select pi()*@radio*@radio as area_circulo;
```

A la hora de crear procedimientos, funciones o disparadores en MySQL, muchas veces tenemos que poner varias sentencias dentro de los mismos (lo que implica poner varias órdenes que acaban en punto y coma). Pues bien, para poderlo hacer tenemos que utilizar las Sentencias BEGIN ..... END.

La sintaxis BEGIN ... END sería:

```
BEGIN  
    [lista_sentencias]  
END
```

La lista\_sentencias la componen las distintas órdenes que tenemos que poner dentro del procedimiento, función o disparador y todas acaban en ;.

Al tener que utilizar el ; dentro del BEGIN ... END. Lo primero que tenemos que hacer es cambiar el delimitador del MySQL, para hacer esto utilizamos la orden:

delimiter // o también \d //

justo antes de comenzar a crear el procedimiento, función o disparador.

Y luego colocamos:

delimiter ; o también \d ;

al final del código del procedimiento, función o disparador.

Vamos a ver todos estos detalles en los siguientes apartados de la unidad de trabajo.

Los procedimientos almacenados es una nueva funcionalidad de la versión de MySQL 5.0. Un procedimiento almacenado es un conjunto de comandos SQL que pueden almacenarse en el servidor. Una vez que se hace, los clientes no necesitan relanzar los comandos individuales, sino que pueden en su lugar referirse al procedimiento almacenado.

Teoría sobre este tema la podemos encontrar en el manual a partir de la página 1023.

Para crear un procedimiento utilizamos la siguiente sintaxis:

CREATE PROCEDURE nombre\_procedimiento ([IN, OUT, INOUT])

BEGIN ... Comandos SQL validos.... END.

Para llamar un procedimiento utilizaremos la orden:

CALL nombre\_procedimiento()

Para ver los procedimientos creados en el servidor:

SHOW PROCEDURE STATUS

Para borrar un procedimiento:

DROP PROCEDURE nombre\_procedimiento;

2.1.- Vamos a crear de momento una base de datos dentro del MySQL llamada PROCEDIMIENTOS, que utilizaremos para aquellos procedimientos para los que no tengamos una base de datos asignada para crearlos.

## **EJEMPLOS DE PROCEDIMIENTOS SIN PARÁMETROS:**

**3.- Crear un procedimiento, llamado area\_circulo, que nos halle el área de un círculo dependiendo del valor de la variable @radio a la que le asignamos un valor dentro del código del mismo procedimiento.**

\d//

CREATE PROCEDURE area\_circulo()

```
BEGIN
SET @radio=15;
SELECT PI()*@radio*@radio AS AREA_CIRCULO;
END
//
\d;
CALL area_circulo();
```

4.- Crear un procedimiento, llamado `borrar_clientes`, que nos permita borrar todos los registros de la tabla `clientes` de la base de datos `concesionario`, cuyo código cliente es mayor o igual a 880.

\d//

```
CREATE PROCEDURE borrar_clientes()
-> BEGIN
-> DELETE FROM clientes
-> WHERE codigo_cliente >= 880;
-> END
-> //
```

```
mysql> call borrar_clientes();
Query OK, 0 rows affected (0.00 sec)
```

5.- Crear un procedimiento, llamado `aleatorio`, que nos muestre un número aleatorio del 1 al 100. Utilizar las funciones `RAND()` y `CEILING(X)` (Retorna el entero más pequeño no menor a X.)

DELIMITER //

```
CREATE PROCEDURE aleatorio()
BEGIN
-- Generar un número aleatorio entre 1 y 100
DECLARE numero_aleatorio INT;
SET numero_aleatorio = CEILING(RAND() * 100);

-- Mostrar el número aleatorio
SELECT numero_aleatorio AS NumeroAleatorio;
END //
```

DELIMITER ;

```
mysql> call aleatorio();
+-----+
| NumeroAleatorio |
+-----+
|          41      |
+-----+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

**6.- Crear un procedimiento, llamado primitiva, que nos muestre 6 números comprendidos entre 1 y 49 (ambos inclusive)**

**DELIMITER //**

**CREATE PROCEDURE primitiva()  
BEGIN**

**-- Declarar variables**

**DECLARE num\_aleatorio INT;**

**DECLARE i INT DEFAULT 1;**

**DECLARE numeros\_generados TEXT DEFAULT "";**

**-- Bucle para generar 6 números aleatorios únicos**

**WHILE i <= 6 DO**

**SET num\_aleatorio = CEILING(RAND() \* 49); -- Generar un número entre 1 y 49**

**-- Verificar si el número ya está en la lista**

**IF FIND\_IN\_SET(num\_aleatorio, numeros\_generados) = 0 THEN**

**-- Agregar el número a la lista y avanzar al siguiente**

**SET numeros\_generados = CONCAT(numeros\_generados, ',', num\_aleatorio);**

**SET i = i + 1;**

**END IF;**

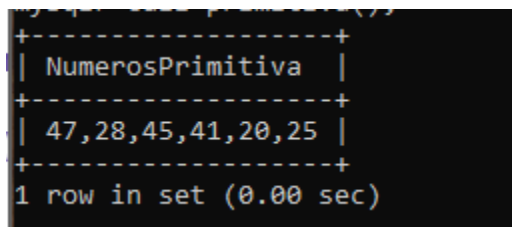
**END WHILE;**

**-- Mostrar los 6 números generados**

**SELECT SUBSTRING(numeros\_generados, 2) AS NumerosPrimitiva; -- Eliminar la coma inicial**

**END //**

**DELIMITER ;**



```
+-----+
| NumerosPrimitiva |
+-----+
| 47,28,45,41,20,25 |
+-----+
1 row in set (0.00 sec)
```

**DELIMITER //**

**CREATE PROCEDURE primitiva2()  
BEGIN**

**DECLARE numero\_aleatorio1 INT;  
DECLARE numero\_aleatorio2 INT;  
DECLARE numero\_aleatorio3 INT;  
DECLARE numero\_aleatorio4 INT;  
DECLARE numero\_aleatorio5 INT;  
DECLARE numero\_aleatorio6 INT;**

**SET numero\_aleatorio1 = CEILING(RAND() \* 49);  
SET numero\_aleatorio2 = CEILING(RAND() \* 49);  
SET numero\_aleatorio3 = CEILING(RAND() \* 49);  
SET numero\_aleatorio4 = CEILING(RAND() \* 49);  
SET numero\_aleatorio5 = CEILING(RAND() \* 49);  
SET numero\_aleatorio6 = CEILING(RAND() \* 49);**

**-- Mostrar el número aleatorio**

**SELECT numero\_aleatorio1 AS NumeroAleatorio1, numero\_aleatorio2 AS  
NumeroAleatorio2, numero\_aleatorio3 AS NumeroAleatorio3, numero\_aleatorio4 AS  
NumeroAleatorio4, numero\_aleatorio5 AS NumeroAleatorio5, numero\_aleatorio6 AS  
NumeroAleatorio6 ;  
END //**

**DELIMITER ;**

```
mysql> call primitiva2();
```

| NumeroAleatorio1 | NumeroAleatorio2 | NumeroAleatorio3 | NumeroAleatorio4 | NumeroAleatorio5 | NumeroAleatorio6 |
|------------------|------------------|------------------|------------------|------------------|------------------|
| 30               | 42               | 19               | 20               | 40               | 44               |

```
1 row in set (0.00 sec)
```